

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №9

по дисциплине «Естественно-языковой интерфейс интеллектуальных
систем»

на тему «Разработка автоматизированной системы распознавания речи
(вариант 1)»

Выполнили студенты гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Крапивин Ю. Б.

Минск 2026

Цель: освоить на практике основные принципы создания систем анализа и распознавания речи.

Задание (вариант 1): разработать автоматизированную систему анализа (распознавания) речи со следующей комбинацией признаков варианта:

- **Поддерживаемые языки** – английский;
- **Предметная область** – научные статьи по computer science.

Система анализа должна обеспечивать следующие минимальные возможности:

- задание списка операций, на которые система может реагировать;
- организацию автоматической реакции системы на речевой сигнал с уведомлением пользователя о происходящем;
- настройки (выбор) естественного языка.

В отчёт включены: описание структуры разработанной системы, технологий и методик; основные алгоритмы и принципы реализации компонент системы (блок-схемы алгоритмов); результаты тестирования; результаты анализа полученных данных и предложения по улучшению работы системы.

Распознавание речи реализовано как продолжение ЛР8 и как ещё один способ управлять уже существующей информационно-поисковой системой (ЛР1–ЛР4): голосом можно продиктовать поисковый запрос, перейти в любой раздел, озвучить открытый документ научной статьи или остановить озвучивание. Распознавание выполняет локальная нейросетевая модель Whisper (библиотека faster-whisper), без обращения к облачным сервисам (см. раздел «Технологии и методики»). Основным языком распознавания – английский (вариант 1); дополнительно, сверх обязательного по варианту, поддержан русский язык.

Структура разработанной системы

Распознавание речи реализовано как новый сценарий использования поверх архитектуры ЛР1–ЛР8 (см. docs/ARCHITECTURE.md). Тяжёлые зависимости (модель Whisper, среда исполнения CTranslate2) изолированы в том же **speech-service**, который появился в ЛР8 для синтеза речи: оба направления – «текст → речь» и «речь → текст» – опираются на локальные нейросетевые модели и одинаковые ограничения по ресурсам контейнера, поэтому отдельного микросервиса для распознавания не потребовалось. Новыми стали эндпоинты и слой сопоставления фраз с командами в **api**, таблица команд в БД и голосовой режим в **frontend**. Структура системы

на уровне контейнеров (C4, уровень 2) показана на рисунке 1; компоненты и связи, добавленные в этой лабораторной работе, выделены зелёным цветом.

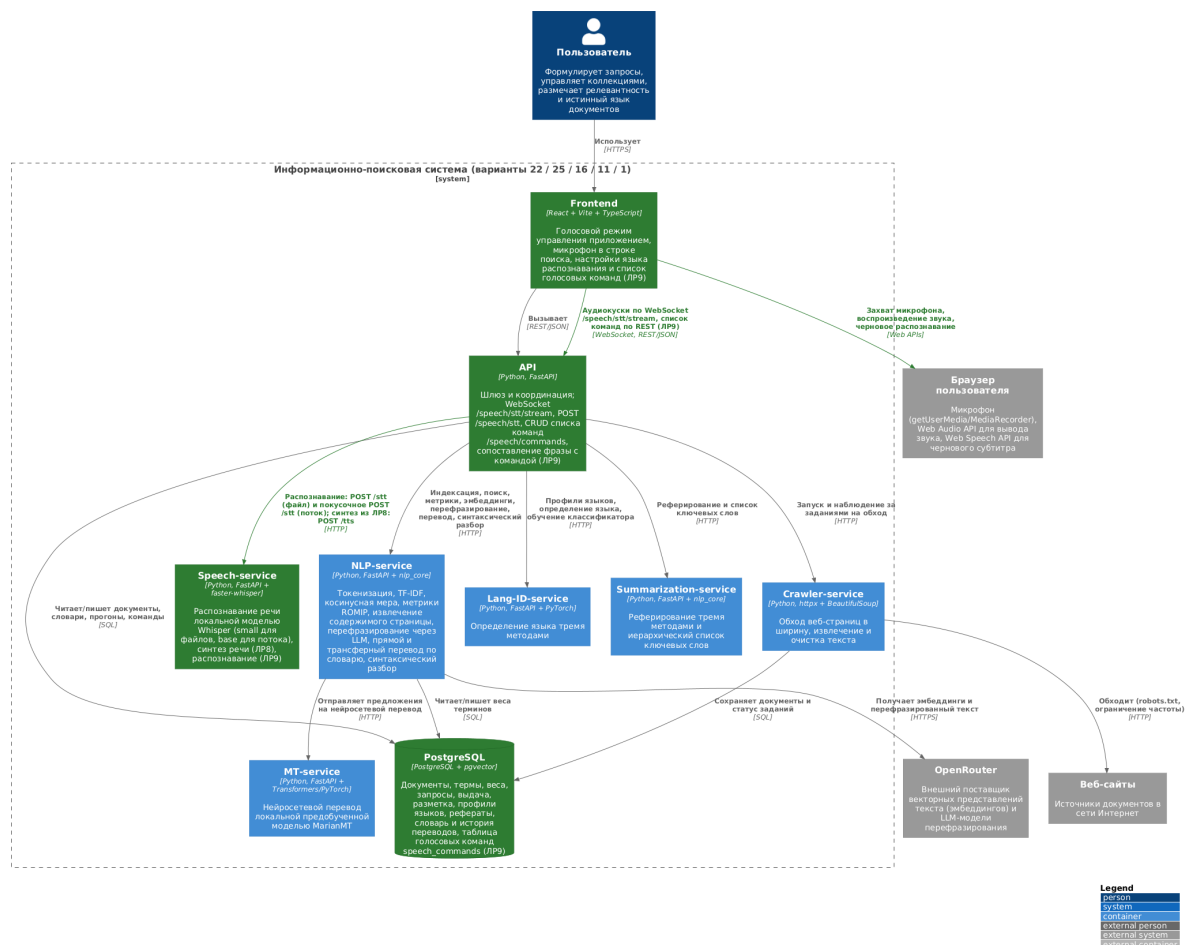


Рисунок 1 – Диаграмма контейнеров разработанной системы (нотация C4); зелёным – новое в LP9

Ответственность компонентов распределена следующим образом.

- **speech-service** (Python, FastAPI, faster-whisper) – эндпоинт `POST /stt` принимает аудиофайл (не более 25 МиБ), язык, подсказку-контекст (`initial_prompt`), флаги фильтра тишины (`vad_filter`) и выбора модели, а возвращает текст, определённый язык и время обработки. Используются две модели: **small** для распознавания записей целиком и более лёгкая **base** для потока, чтобы куски аудио не отставали от реального времени. Обе загружаются при старте контейнера в фоновом потоке, чтобы первое обращение пользователя не ждало загрузки модели;

- **api** (Python, FastAPI) – `POST /speech/stt` (запись целиком), `WebSocket /speech/stt/stream` (поток: клиент присылает аудиокус-ки, сервер отвечает частичными результатами), `CRUD-эндпоинты /speech/commands` для списка команд и сопоставление распознанной фразы с командой. Число одновременных обращений к **speech-service** ограничено семафором (по умолчанию 4), чтобы несколько сеансов не превысили

бюджет ядер контейнера; один кусок потока обрабатывается не более 20 с;

- **postgres** – таблица **speech_commands** (список операций, на которые реагирует система);
- **frontend** (React, TypeScript) – захват микрофона, разбиение речи на фразы, отображение реакции и выполнение действия команды. Компоненты перечислены в таблице 1.

Таблица 1 – Компоненты интерфейса, реализующие распознавание речи

Компонент	Назначение
<code>useLiveSttStream</code>	Захват микрофона, запись кусками по 3,5 с, обмен с сервером по WebSocket, аварийное завершение сеанса
<code>SpeechModeContext</code> , <code>AssistantMicButton</code>	Голосовой режим: постоянное прослушивание команд из любого раздела приложения, кнопка-микрофон в правом нижнем углу, уведомления
<code>VoiceInputButton</code>	Микрофон в строке поиска и на вкладке «Распознавание речи» страницы «Настройки»: диктовка запроса с выполнением команд
<code>commandDispatch.ts</code> , <code>speechActions.ts</code>	Выполнение действия найденной команды; названия и подсказки действий
<code>matchCommand.ts</code>	Сопоставление фразы с командой на стороне браузера (зеркало серверной функции)
<code>micVolumeMeter.ts</code> , <code>liveSpeechRecognition.ts</code>	Уровень громкости микрофона (пульсация кнопки, обнаружение пауз); черновой субтитр Web Speech API
Вкладки «Распознавание речи» и «Команды» страницы «Настройки»	Выбор языка, фразы активации, проверка микрофона; добавление, изменение и удаление команд

Структура базы данных

Для хранения списка операций добавлена одна таблица **speech_commands**, устроенная так (миграции 0016, 0018 и 0023):

- **id** – первичный ключ; порядок **id** определяет приоритет при сопоставлении (см. далее);
- **phrase** (до 200 символов) – ключевая фраза, которую ищут в сказанном;
- **action** (до 50 символов) – идентификатор действия из фиксированного набора (см. таблицу 2): пользователь выбирает действие из списка, а не вводит произвольный код, поэтому добавить «несуществующую» команду нельзя;
- **language** (до 10 символов, по умолчанию **en**) – язык фразы: команда срабатывает только тогда, когда выбран тот же язык распознавания;
- **is_active** – флаг, позволяющий отключить команду, не удаляя её;
- **created_at** – время создания.

Миграции сразу заполняют таблицу начальным набором из 13 команд на каждом из двух языков (английском и русском), покрывающим все действия (см. таблицу 2); этот набор пользователь затем расширяет и меняет на вкладке «Команды» страницы «Настройки» без переразвёртывания системы.

Задание списка операций

Требование «задание списка операций, на которые система может реагировать» реализовано как редактируемый список команд. Каждая команда – это пара «фраза → действие»; действий 13 (таблица 2). Список доступен на вкладке «Команды» страницы «Настройки»: команды добавляются, переименовываются, переключаются между английским и русским языком, отключаются и удаляются; после изменения список сразу перечитывается голосовым режимом.

Таблица 2 – Действия системы и начальный набор фраз

Действие	Английская фраза	Русская фраза
Перейти к поиску и выполнить запрос (сказанное как запрос)	search for ...	найди ...
Очистить поле запроса	clear search	очисти запрос
Озвучить открытый документ (или первый доступный на странице текст: реферат, перевод)	read this	прочитай это
Остановить озвучивание	stop reading	хватит читать
Открыть «Кроулинг»	go to crawling	перейди к краулингу
Открыть «Коллекции»	go to collections	перейди к коллекциям
Открыть «Поиск» без запроса	go to search	перейди к поиску
Открыть «Метрики»	go to metrics	перейди к метрикам
Открыть «Язык»	go to language identification	перейди к определению языка
Открыть «Реферирование»	go to summarization	перейди к реферированию
Открыть «Перевод»	go to translation	перейди к переводу
Открыть «Настройки»	go to settings	перейди к настройкам
Открыть «Справку»	go to help	перейди к справке

Дополнительно к списку команд в настройках задаётся необязательная *фраза активации*: если она не пуста, система реагирует только на фразы, в которых эта фраза есть, и отбрасывает всё, что было сказано до неё (например, для фразы «computer» команда произносится как «computer, go to metrics»). Это защищает от случайных срабатываний в шумном помещении, когда голосовой режим слушает постоянно.

Технологии и методики

Выбор технологии распознавания. Методичка предлагает Sphinx4, Microsoft Speech API, Google Speech API, React-speech-recognition (Web Speech API) и нейросетевые библиотеки. Web Speech API в Chrome отправляет звук на серверы Google и работает только в некоторых браузерах; Sphinx4 и другие классические акустические модели заметно уступают нейросетевым по точности на произвольной лексике, а научные статьи по computer science полны редких терминов; облачные API требуют ключей и передачи речи третьей стороне. Поэтому выбрана нейросетевая модель **Whisper** в реализации **faster-whisper** 1.2.1 (среда CTranslate2, квантование INT8, работа на CPU), запускаемая локально в контейнере. Web Speech API не отброшен полностью, а используется для одной вспомогательной цели: показывать пользователю черновой субтитр сказанного по словам прямо во время речи, пока серверная модель ещё обрабатывает кусок. Текст законченной фразы берётся из субтитра, если он доступен (браузеры Chrome и Edge), а иначе склеивается из серверных результатов по кускам.

Две модели. Точность растёт с размером модели, но растёт и время обработки. Для записей целиком, где пользователь готов подождать доли секунды, используется **small**; для потока – **base**, у которой обработка куска идёт заметно быстрее (см. таблицу сравнения моделей в разделе «Результаты тестирования»).

Подсказка-контекст. Каждый следующий кусок потока распознаётся с параметром **initial_prompt**, равным последним 200 символам уже распознанного текста: модель учитывает предыдущий контекст, поэтому фраза, разорванная границей кусков, не теряет согласования, а научная лексика распознаётся стабильнее.

Фильтр тишины. В потоке включён **vad_filter** (детектор речевой активности Silero, встроенный в faster-whisper): куски, в которых никто не говорит, возвращают пустой результат, а не «галлюцинации» Whisper на шуме, для которых модель склонна выдавать случайные фразы.

Выбор естественного языка. Пользователь выбирает язык распознавания на вкладке «Распознавание речи» (английский или русский); выбор передаётся в Whisper как явная подсказка языка, чтобы модель не тратила время и точность на его определение, и одновременно определяет, какие команды считаются действующими. Для варианта 1 основной язык – английский; русский добавлен как второй язык и потребовал только русской озвучки списка команд (см. таблицу действий) – благодаря тому, что модель Whisper многоязычная, а список команд хранится в БД.

Основные алгоритмы

Общая схема потокового распознавания. Последовательность обмена сообщениями при потоковом распознавании показана на рисунке 2. Браузер записывает звук кусками по 3,5 с (запись останавливается и запускается заново, чтобы каждый кусок был независимым и самодостаточным аудиофайлом, который Whisper способен декодировать), отправляет их по WebSocket, а в ответ получает текст каждого куска.

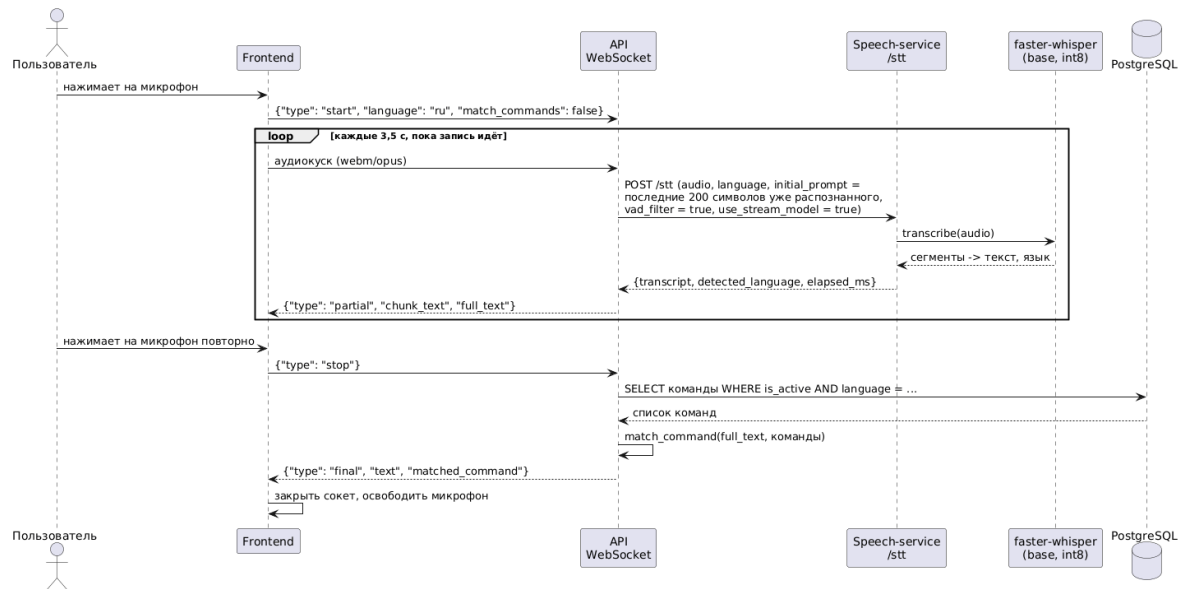


Рисунок 2 – Потоковое распознавание речи по WebSocket

Разбиение речи на фразы по паузам. Автоматическая реакция системы на речь требует решить, когда фраза закончилась. Разбиение на равные интервалы времени (например, по 3,5 с) режет длинные фразы на части, поэтому команда, попавшая на границу двух кусков, теряется. Вместо этого система копит распознанный текст и оценивает *паузу по громкости микрофона*: уровень определяется как среднеквадратичное значение сигнала (RMS), измеряемое через **AnalyserNode** Web Audio API и масштабируемое коэффициентом 4 до диапазона от 0 до 1. Пока уровень выше порога 0,06, пользователь считается говорящим. Накопленный текст считается законченной фразой, когда после речи наступила тишина более 1,8 с (и субтитр Web Speech API не менялся последние 0,5 с) либо фраза длится дольше 12 с (рисунок 3). Если браузер не позволяет измерить громкость, каждый серверный кусок считается отдельной фразой.

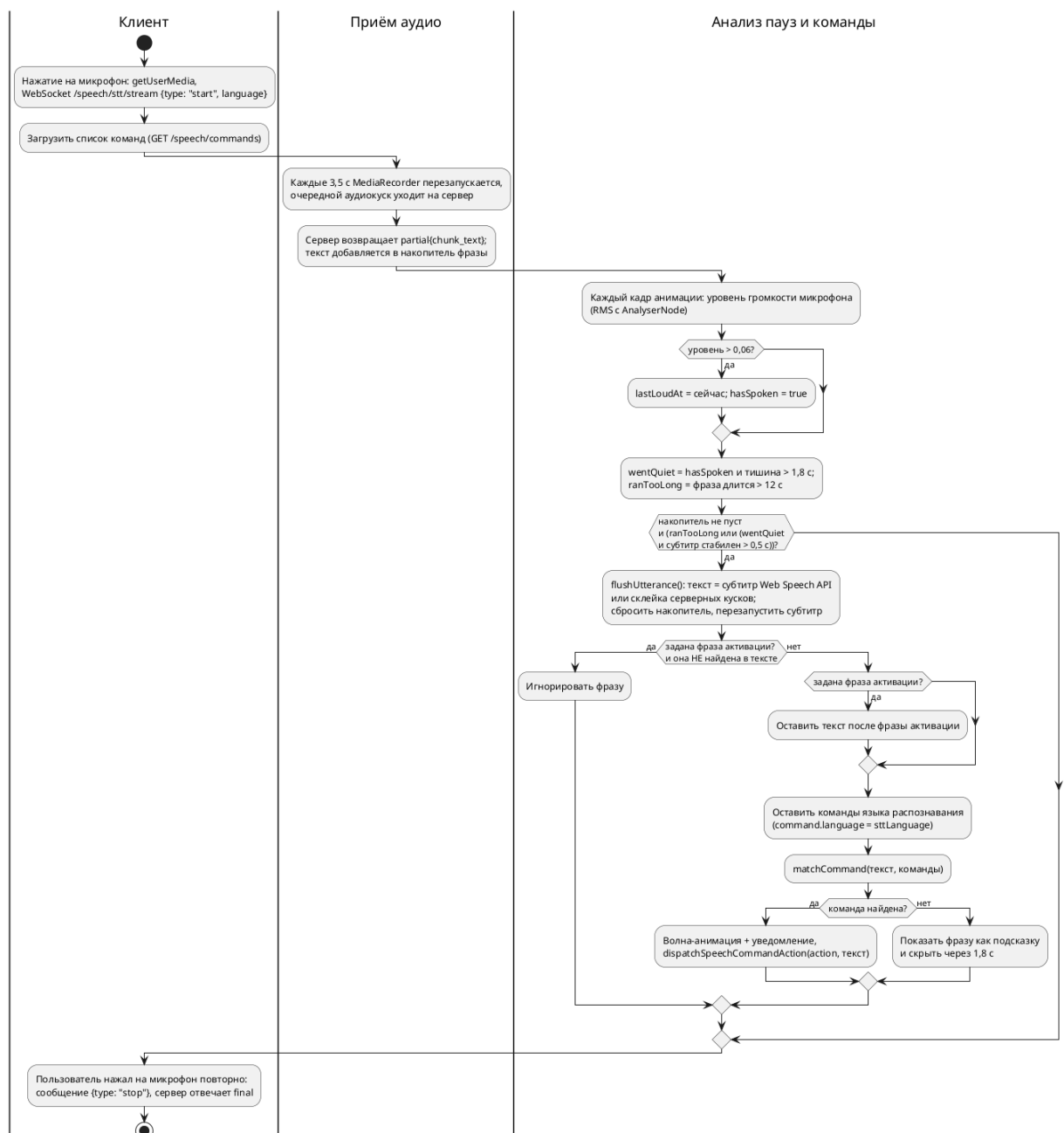


Рисунок 3 – Непрерывное прослушивание: разбиение на фразы и реакция на команду

Сопоставление фразы с командой. Алгоритм (рисунок 4) намеренно прост и предсказуем: текст и фраза команды нормализуются (знаки препинания заменяются пробелами, регистр приводится к нижнему), после чего команда считается сработавшей, если её фраза *входит* в текст как подстрока; из нескольких подходящих выбирается первая по порядку id. Подстрока, а не точное совпадение, потому что Whisper добавляет к фразе знаки препинания и слова-паразиты («Okay, go to settings.»), а команда «search for ...» по определению содержит произвольный хвост. Фразы начального набора подобраны так, чтобы ни одна не была подстрокой другой (иначе более короткая перехватила бы более длинную). Цена простоты

– ложные срабатывания на обычных предложениях, содержащих фразу команды (см. «Результаты тестирования» и «Анализ результатов»).

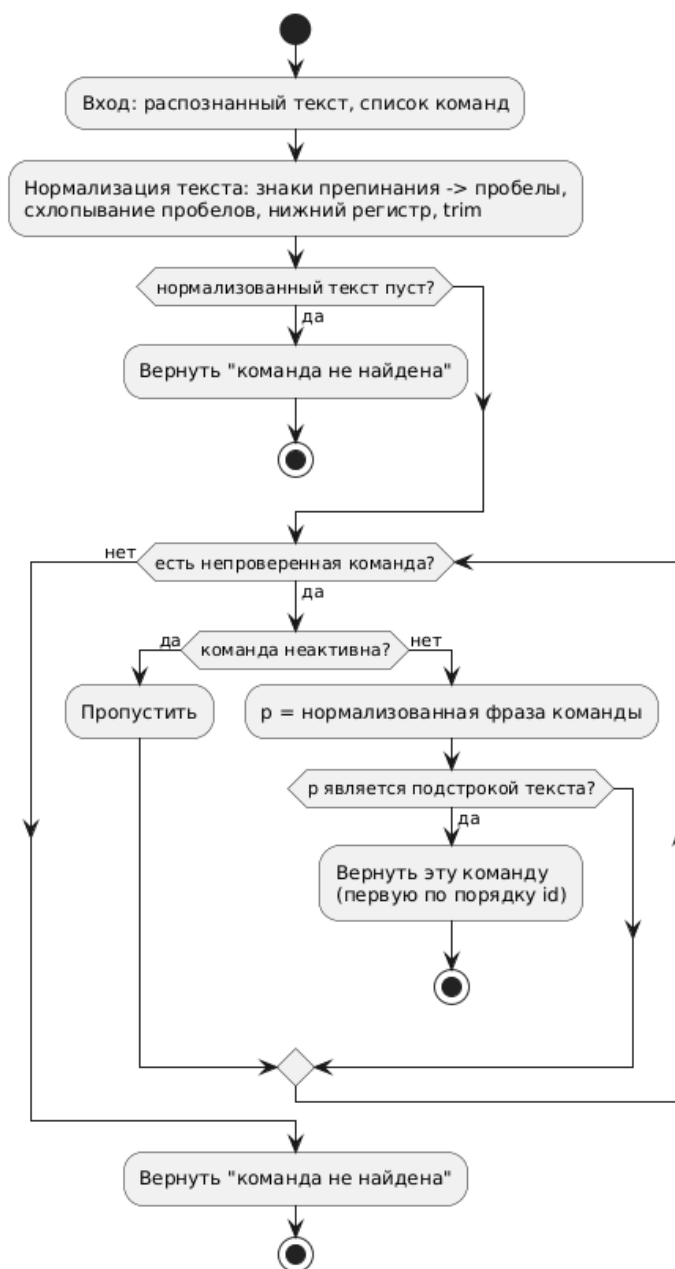


Рисунок 4 – Сопоставление фразы с командой

Сопоставление выполняется и на сервере (для потока – по сообщению **stop**, для записи целиком – в ответе **POST /speech/stt**), и на клиенте (функция-зеркало **matchCommand.ts**): клиентское сопоставление позволяет реагировать на каждую фразу, не дожидаясь завершения всего сеанса, а серверное – отвечает за режим «записал – получил результат». Обе реализации отбирают только команды выбранного языка распознавания.

Действие и уведомление пользователя. Когда команда найдена, система (а) проигрывает по экрану зелёную волну от кнопки

микрофона (анимация длительностью 1,4 с), (б) выполняет действие (`dispatchSpeechCommandAction`): переход на страницу, подстановка запроса, очистка поля, озвучивание документа или остановка озвучивания, (в) показывает уведомление с названием сработавшей команды. Пока пользователь говорит, рядом с кнопкой показывается распознанный текст, а сама кнопка пульсирует в такт громкости голоса; после завершения фразы, не совпавшей ни с одной командой, текст остаётся на экране ещё 1,8 с, чтобы пользователь видел, что система его услышала, но команду не поняла.

Устойчивость непрерывного сеанса. Сеанс может идти неограниченно долго, поэтому в нём предусмотрено несколько защит. Единичный сбой обработки куска считается временным и молча пропускается; только три сбоя подряд останавливают прослушивание с сообщением об ошибке. Потеря микрофона (например, его отключили) и обрыв соединения распознаются и сообщаются пользователю. Повторный запуск не создаёт второго параллельного сеанса. Задержанные остановки предыдущего сеанса отменяются при запуске нового, чтобы таймер завершения старого сеанса не закрыл только что начавшийся.

Способы использования и пользовательский интерфейс

Реализованы три способа управлять системой голосом:

- **Голосовой режим** – кнопка-микрофон в правом нижнем углу любой страницы. После включения система постоянно слушает и выполняет любую команду из списка; так можно, не касаясь мыши, перейти в нужный раздел, озвучить открытый документ или остановить озвучивание;

- **Микрофон в строке поиска** – диктовка поискового запроса: сказанная фраза попадает в поле, а если она совпала с командой, выполняется команда (например, «clear search» очищает поле). Микрофон слушает непрерывно, пока пользователь не нажмёт на него ещё раз; каждая новая фраза заменяет предыдущую;

- **Проверка распознавания** на странице «Настройки» – то же поведение без выполнения команд, для проверки микрофона и качества распознавания выбранного языка.

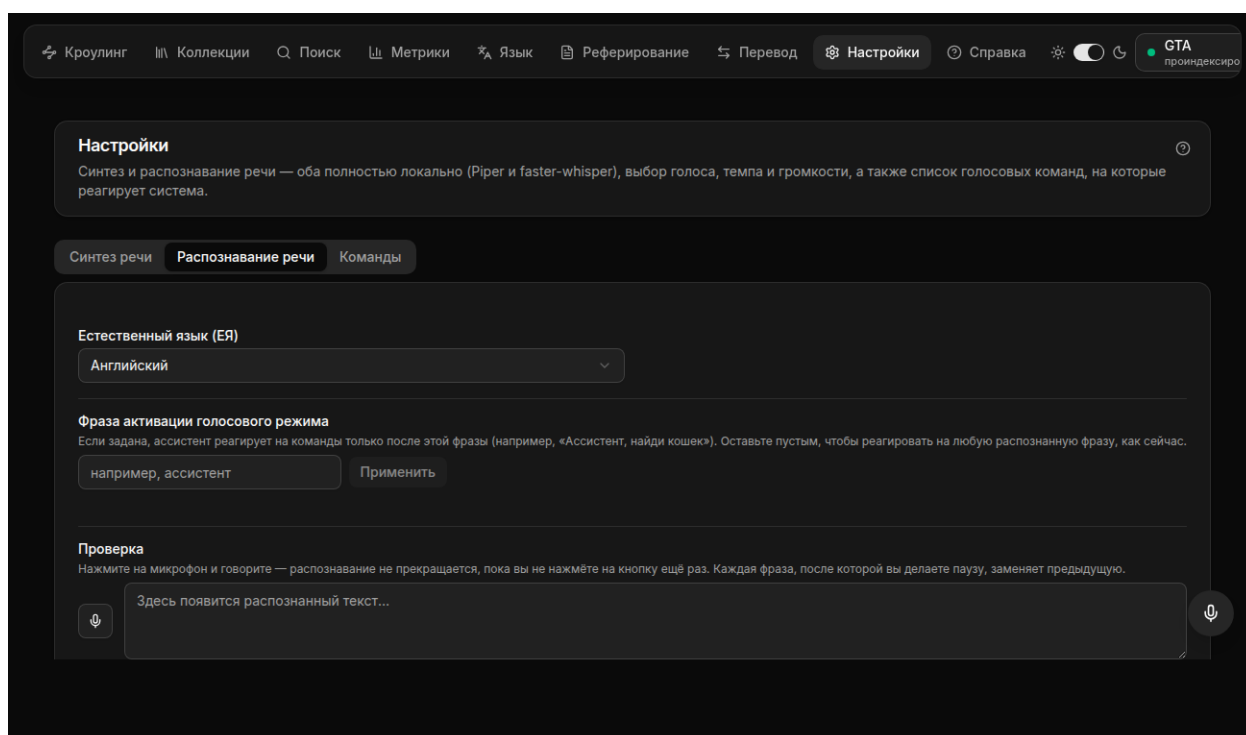


Рисунок 5 – Вкладка «Распознавание речи»: выбор языка, фраза активации, проверка микрофона

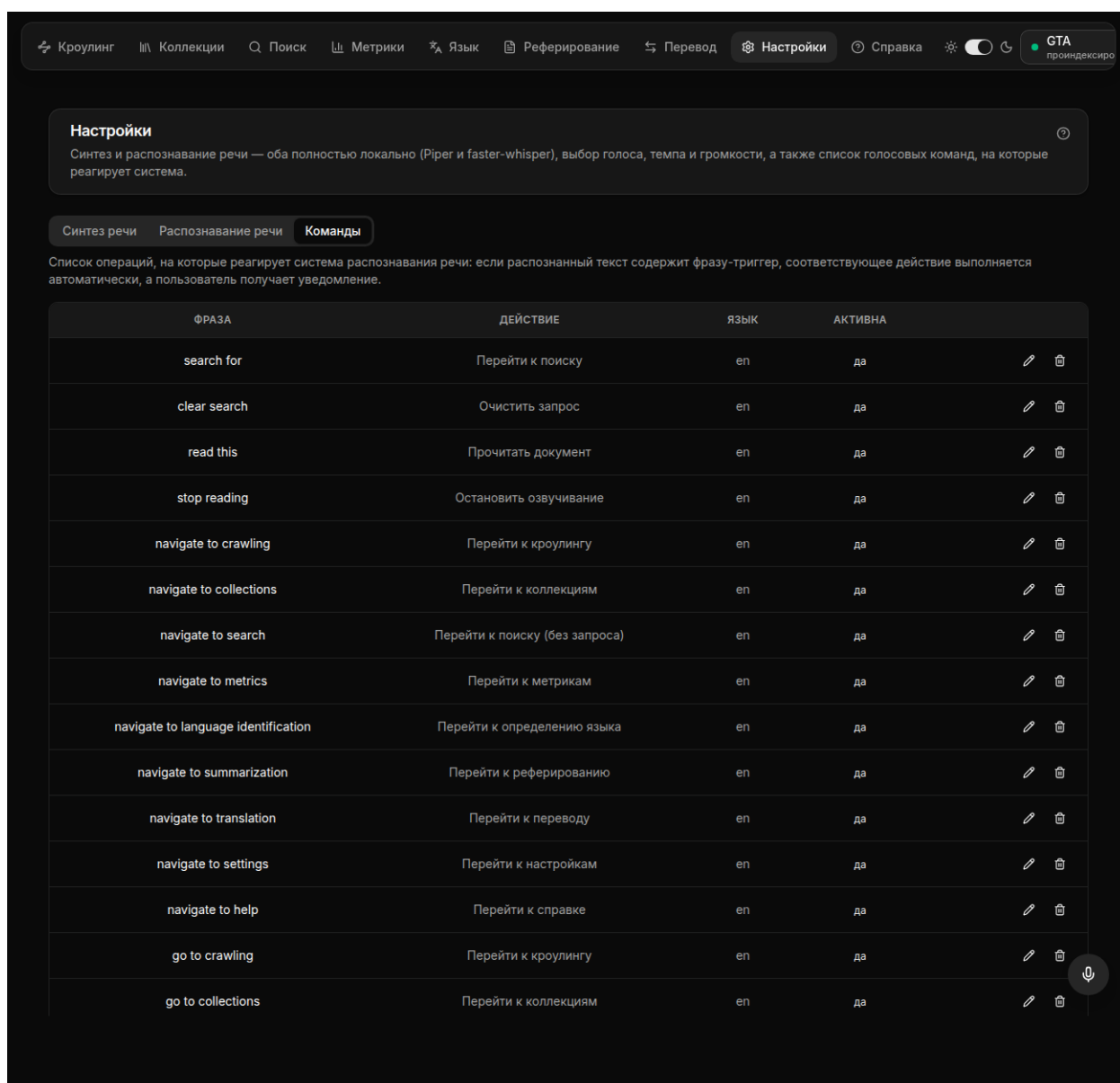


Рисунок 6 – Вкладка «Команды»: список операций, на которые реагирует система (добавление, изменение, отключение и удаление)

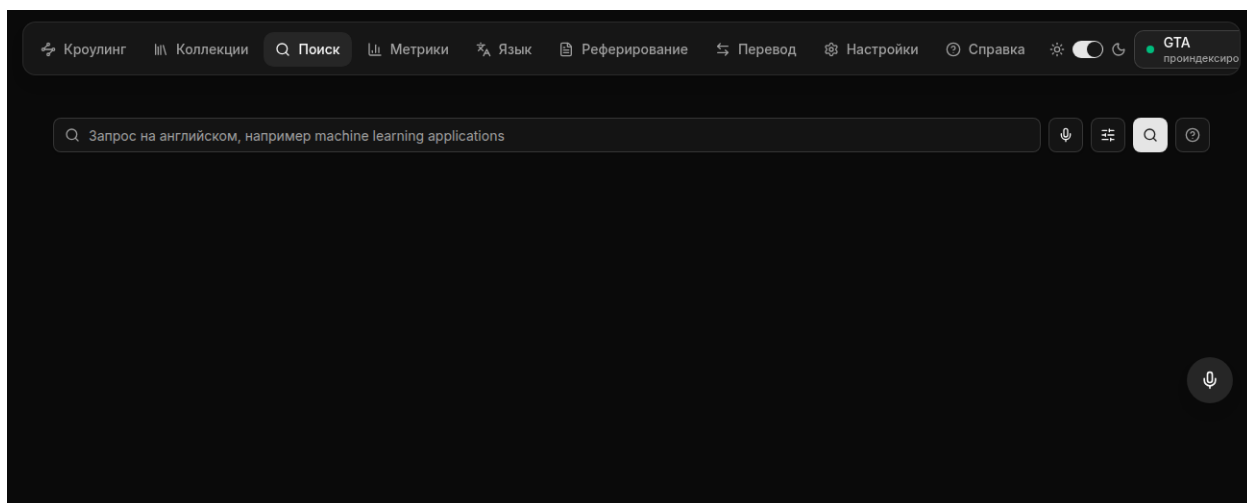


Рисунок 7 – Страница «Поиск»: микрофон рядом со строкой запроса и кнопка голосового режима в правом нижнем углу

Результаты тестирования

Методика. Автоматическая проверка распознавания требует записанной речи, а искать дикторов для сотен реплик невозможно, поэтому тестовая речь *синтезирована* голосами Piper из LP8 и подана на вход POST /speech/stt (модель small) – тот же эндпоинт, которым пользуется интерфейс, вместе с подстановкой команд из БД. Английские реплики озвучены голосами Amy и Ryan (США) и Alba (Великобритания), русские – голосами Irina и Denis (Piper, ru_RU; в систему они не устанавливались, использовались только для подготовки тестовых записей). Наборы:

- *команды* – 13 английских и 13 русских фраз из таблицы 2, каждая двумя голосами (по 26 записей);

- *предложения* – 10 английских предложений из области варианта (научные статьи по computer science) тремя голосами (30 записей) и 5 русских двумя голосами (10 записей);

- *не команды* – обычные предложения, которые не должны вызывать никакого действия (7 английских, включая два трудных случая, и 5 русских, каждое двумя голосами).

Метрики: доля правильно найденных команд (найденная команда совпала с ожидаемой), доля ошибочных слов WER (расстояние Левенштейна на уровне слов между исходной фразой и распознанным текстом, нормализованное по регистру и знакам препинания, на общем числе слов эталона), число ложных срабатываний, время обработки. Контейнер был ограничен четырьмя ядрами CPU.

Ограничение методики. Синтезированная речь чище живой: у неё нет акцента, оговорок, дыхания и фонового шума микрофона. Поэтому полученные ниже оценки надо рассматривать как *верхнюю границу* качества на реальной речи, а не как её прогноз; измерение на записанной речи – первое из предложений по улучшению. Русские записи озвучены голосами Piper, которые сами по себе менее естественны английских, так что часть русских ошибок может объясняться не распознаванием, а качеством тестовой речи.

Итоги по наборам приведены в таблице 3.

Английский язык (основной по варианту). Из 26 команд не найдены две, и обе – из-за ошибки распознавания, а не сопоставления: «clear search» распознано как «CLEAR CERT», «go to settings» – как «Build the settings.». Остальные фразы распознаны дословно, в том числе «search for machine learning applications» с произвольным хвостом. Предложения из предметной области распознаются с WER 2,6 %: все ошибки – на редких терминах («lemmatization» → «limitization», «hyperplane» → «hyperblane») и на артикле в начале фразы. Единственные ложные срабатывания – предложение «The search for optimal hyperparameters is expensive»: оно содержит фразу «search for», а алгоритм сопоставления по подстроке не различает

Таблица 3 – Результаты распознавания синтезированной речи (модель **small**)

Набор	Записей	Результат
Английские команды	26	найдено верно 24 (92,3 %); Amy 13/13, Ryan 11/13; WER 3,8 %
Английские предложения (computer science)	30	WER 2,6 % (Amy 2,0, Ryan 2,0, Alba 3,9); без ошибок 24 из 30
Английские не команды	10	ложных срабатываний 0
Английские «трудные» предложения (содержат фразу команды)	4	ложных срабатываний 2 (оба раза – «The search for optimal hyperparameters. . . »)
Русские команды	26	найдено верно 16 (61,5 %); Irina 10/13, Denis 6/13
Русские предложения	10	WER 27,1 % (Irina 22,9, Denis 31,4)
Русские не команды	10	ложных срабатываний 0

команду и обычное предложение с теми же словами. Второе трудное предложение («Please go to the next section. . . ») не вызвало срабатывания, так как ни одна команда не состоит из слов «go to» без продолжения.

Русский язык (дополнительный). Распознавание заметно хуже: верно найдено 16 команд из 26 (61,5 %), WER предложений – 27,1 %. Разрыв объясняется двумя причинами. Во-первых, русская морфология: ошибка в одной букве окончания («коллекции» вместо «коллекциям», «реферировании» вместо «реферированию») ломает точное сопоставление подстроки, хотя смысл фразы понятен. Во-вторых, узкоспециальные слова, которых нет в обычной речи, распознаются с ошибками («краулингу» → «Каулинбу», «термов» → «темы», «булев» → «були»). Голос Denis дал 6 из 13 против 10 из 13 у Irina, что указывает на влияние качества тестовой речи. Ложных срабатываний на русских не командах не было.

Две модели распознавания. Модель **base**, которой распознаётся поток, сравнена с **small** на тех же английских записях (таблица 4). **base** втрое быстрее, но почти вдвое чаще ошибается. Для команд разница особенно заметна: время обработки короткой фразы (1,2 с речи) у **small** – 2,7 с, то есть дольше самой фразы, у **base** – 0,8 с. Заметная часть времени **small** не зависит от длины фразы: модель обрабатывает аудио окнами по 30 с, поэтому короткая команда стоит почти столько же, сколько предложение втрое длиннее (2,7 против 2,4 с).

Таблица 4 – Сравнение моделей распознавания на английских записях

Модель	WER предложений, %	WER команд, %	Время, команда, с	Время, предложение, с
small (записи целиком)	2,6	3,8	2,74	2,42
base (поток)	4,9	7,7	0,84	0,92

Устойчивость к шуму. К десяти английским предложениям (голос

Аму) добавлялся гауссов шум с заданным отношением сигнал/шум (SNR); результат – в таблице 5. До SNR 5 дБ включительно WER не изменился по сравнению с чистой записью (2,0 %), при 0 дБ (шум по мощности равен речи) вырос до 5,9 %. Такая устойчивость – следствие того, что Whisper обучен на очень разнообразных данных, однако равномерный гауссов шум легче реального (музыка, чужая речь, эхо помещения), поэтому реальный запас качества меньше.

Таблица 5 – Влияние шума на точность распознавания (10 английских предложений, голос Аму, модель `small`)

Без шума	SNR 20 дБ	SNR 10 дБ	SNR 5 дБ	SNR 0 дБ
2,0 %	2,0 %	2,0 %	2,0 %	5,9 %

Время реакции. Слагаемые задержки от конца фразы до выполнения команды в голосовом режиме: пауза до признания фразы законченной (1,8 с – задаётся константой) плюс время обработки последнего куса потока (около 0,8–0,9 с у модели `base`, см. таблицу 4); распознавание идёт по кускам параллельно с речью, поэтому в реальности последний кусок обычно короче. То есть команда выполняется приблизительно через 2,5–3 с после окончания фразы; заметную долю задержки составляет именно намеренная пауза, а не вычисления.

Автоматические тесты. Слой сопоставления фраз с командами (`test_speech_domain.py`), сессия потока (`test_speech_streaming_domain.py`), службы распознавания и списка команд с базой данных SQLite (`test_speech_application.py`, в том числе проверка того, что команды отбираются по выбранному языку) и маршруты `speech-service` (`test_routes.py`) покрыты тестами; весь набор прошёл без ошибок (18 тестов `api` и 9 тестов `speech-service`). Тесты и измерения выше относятся к серверному распознаванию; в Chrome текст фразы может браться из черновика Web Speech API, качество которого здесь не измерялось. Поведение в браузере (микрофон, пульсация, реакция на команду) автоматическими тестами не покрыто и проверяется вручную.

Анализ результатов и предложения по улучшению

Что показали измерения.

а) *Для английского (вариант 1) качество достаточное.* На чистой речи 92 % команд находятся верно, а все ошибки распознавания приходятся на редкие слова. Дополнительные 8 % потерь – это ошибки самой модели на коротких фразах без контекста: фраза из двух-трёх слов даёт Whisper мало информации.

б) *Русский язык поддержан, но слабее.* Причина – не столько распознавание, сколько жёсткость сопоставления: команда не срабатывает при любом отличии окончания.

в) *Шум почти не влияет* (до 5 дБ), но это оценка по синтезированной речи и гауссову шуму.

г) *Задержка определяется паузой-порогом* (1,8 с) и скоростью модели: **small** обрабатывает даже короткую фразу дольше, чем она длится, поэтому для потока применяется **base** ценой почти двукратного роста ошибок.

д) *Сопоставление по подстроке приводит к ложным срабатываниям* на обычных предложениях, содержащих фразу команды («The search for...»). В голосовом режиме, который слушает постоянно, такое предложение, сказанное вслух рядом с компьютером, перейдёт на страницу поиска.

Ограничения системы.

– Оценки получены на синтезированной, а не живой речи (см. «Ограничение методики»).

– Распознавание работает только в браузерах с MediaRecorder и WebSocket; чернового субтитра нет вне Chrome/Edge (Web Speech API), – но без него система работает, лишь без показа слов по ходу речи.

– Команда – это фраза-подстрока: параметров в командах нет, кроме единственной «search for ...», где параметром служит весь текст.

– Русские команды заданы фиксированной формой окончаний.

– Распознавание идёт на CPU: на четырёх ядрах одновременно обслуживаются не более четырёх сеансов (семафор), остальные ждут.

Предложения по улучшению.

а) *Измерить качество на живой речи:* записать 50–100 реплик нескольких дикторов в реальных условиях (шум помещения, разные микрофоны) и повторить те же измерения.

б) *Нечёткое сопоставление команд:* сравнивать леммы слов (для русского – через морфологический анализатор), либо допускать небольшое расстояние Левенштейна между словами фразы и распознанным текстом; это устранило бы часть русских промахов и ошибки вида «clear cert» → «clear search».

в) *Устранить ложные срабатывания:* требовать, чтобы фраза команды была в начале высказывания («search for X», но не «The search for X»), либо использовать фразу активации по умолчанию (сейчас она необязательна).

г) *Подсказка модели словарём команд:* передавать Whisper в **initial_prompt** названия команд и специальные термины предметной области (краулинг, реферирование, TF-IDF), чтобы редкие слова распознавались увереннее.

д) *Ускорить распознавание команд:* для коротких фраз использовать

модель **base** (с 3-кратным выигрышем во времени) при включённой проверке результата на совпадение с командой, а **small** оставить для диктовки.

е) *Определять язык автоматически* (Whisper это умеет) и выбирать список команд по результату, чтобы пользователю не требовалось переключать язык вручную.

ж) *Голосовое подтверждение действия*: озвучивать сработавшую команду голосом Piper из ЛР8 («Открываю метрики»), замыкая цикл «речь → действие → речь».

Использованные готовые компоненты

Таблица 6 – Библиотеки, готовые данные и программные средства, использованные при реализации ЛР9 (дополнительно к перечисленным в отчётах по ЛР1–ЛР4 и ЛР8)

Компонент	Назначение
faster-whisper 1.2.1 (CTranslate2)	Реализация модели Whisper для распознавания речи на CPU с квантованием INT8; модели small (записи целиком) и base (поток), детектор речевой активности Silero VAD.
Whisper (OpenAI)	Многоязычная нейросетевая модель распознавания речи; используется как английский, так и русский язык.
FastAPI, WebSocket, httpx	Эндпоинты <code>POST /stt</code> , <code>WebSocket /speech/stt/stream</code> , клиент проксирования (используются и в предыдущих лабораторных).
MediaRecorder, getUserMedia	Захват звука микрофона в браузере кусками (<code>webm/opus</code>).
Web Audio API (<code>AnalyserNode</code>)	Измерение уровня громкости микрофона: пульсация кнопки и определение пауз в речи.
Web Speech API (<code>SpeechRecognition</code>)	Только черновой субтитр во время речи; финальный текст фразы приходит с сервера.
SQLAlchemy, Alembic	Модель и миграции таблицы <code>speech_commands</code> (уже использовались).
Piper (ЛР8)	Синтез тестовой речи для автоматизированной проверки распознавания (см. «Результаты тестирования»).

Выводы

В ходе выполнения лабораторной работы реализована автоматизированная система анализа (распознавания) речи по варианту 1 (английский язык, научные статьи по computer science) как продолжение ЛР8 и как ещё один способ управлять существующей информационно-поисковой системой: голосом можно продиктовать поисковый запрос, перейти в любой раздел приложения, озвучить открытый документ и остановить озвучивание. Распознавание выполняет локальная модель Whisper (`faster-whisper`) в том же `speech-service`, что и синтез речи ЛР8, без обращения к облачным сервисам.

Обязательные возможности реализованы следующим образом. *Задание списка операций*: таблица `speech_commands` и вкладка «Команды», на которой пары «фраза → действие» добавляются, изменяются, отключаются и удаляются без перерезвёртывания системы; начальный набор –

13 действий на английском и русском языках. *Автоматическая реакция с уведомлением*: голосовой режим слушает непрерывно, разбивает речь на фразы по паузам (а не на равные интервалы времени, которые режут длинные команды пополам), находит команду и выполняет её, показывая распознанный текст, пульсацию микрофона в такт голосу, зелёную волну и уведомление о сработавшей команде. *Выбор естественного языка*: английский или русский, причём выбор определяет не только язык распознавания, но и то, какие команды считаются действующими.

Измерения на синтезированной речи показали для основного языка варианта (английского) 92,3 % верно найденных команд и WER 2,6 % на предложениях из предметной области; шум с отношением сигнал/шум до 5 дБ на точность не повлиял. Русский язык, добавленный сверх обязательно по варианту, распознаётся хуже (61,5 % команд, WER 27,1 %): причина – не только качество распознавания, но и жёсткое сопоставление по подстроке, не терпящее ни одного изменения окончания. Модель **base** в потоке втрое быстрее **small**, но почти вдвое чаще ошибается.

Проверка выявила и слабые места, описанные в разделе «Анализ результатов и предложения по улучшению»: ложные срабатывания на обычных предложениях, содержащих фразу команды («The search for...»), чувствительность русских команд к окончаниям и то, что все оценки получены на синтезированной, а не живой речи. Для каждого из них предложено решение; наиболее важные – измерение на записанной речи и нечёткое сопоставление команд.

В процессе проверки также найден и исправлен дефект, который проявлялся как «ошибки через некоторое время» на микрофонах поиска и проверки: отложенный таймер остановки предыдущего сеанса закрывал только что начавшийся, если новая запись запускалась в течение 22 с после остановки прежней. Теперь таймеры привязаны к своему сеансу и отменяются при запуске нового, а сами микрофоны поиска и проверки слушают непрерывно, как и голосовой режим, пока пользователь не остановит их сам.